Team Project Report - Phase 3

Phase 3 Summary

Team name: dwendwen

Members (names and Andrew IDs):

Qing Deng (qingd)

Wangshu Li (wangshul)

Jiarui Zhang (jiaruiz)

Performance data and configurations

Please note down the following information for your service in the live test.

Describe your web-tier configuration (ECS Fargate/EKS Fargate/EKS managed node group, what configuration):

EKS with managed node groups

9 c6a.large instances, each with 20 GB root volume

Describe your storage-tier configuration (which service, what configuration):

RDS (MySQL)

two db.r6g.large instances with 50 Gb each

Cost per hour of the entire system:

EKS: 0.1

Instances: (0.0765 + 20 * 0.1/720) * 9 = 0.7135

Load Balancer: 0.0225

(Web-tier total: 0.836)

RDS(with volume): 0.22 * 2

Total: 0.836 + 0.44 = $1.276/hour

Your team's overall rank on the live test scoreboard: 36

Live test results for each microservice:

	M1	M2	M3
score	14.24	25	23.12
submission id	730822	730823	730824
throughput	61701.39	55425.22	9246.70
latency	7.74	7.95	51.40
correctness	100	88.36	99.88
error	0	0	0
rank	62	20	36

Rubric

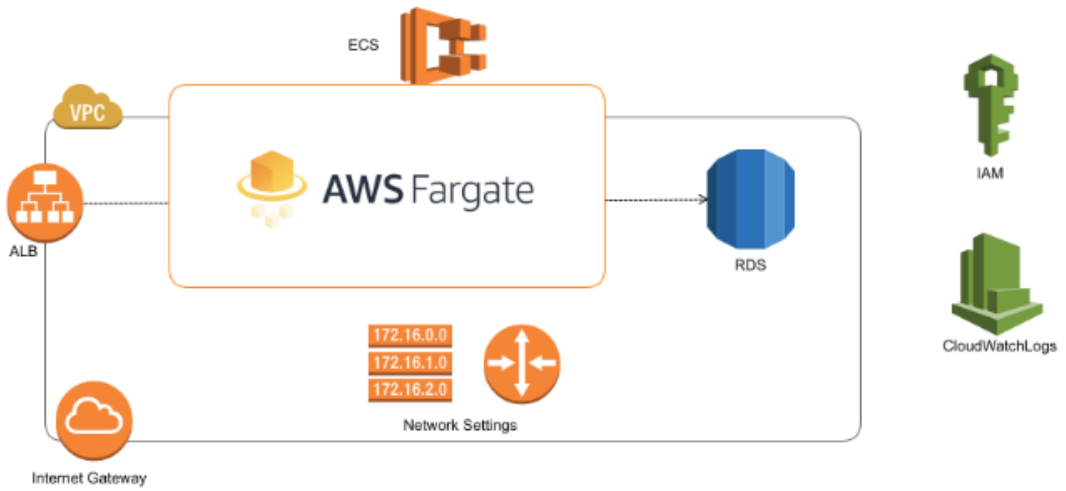
- Each unanswered bullet point = -4%
- Each unsatisfactory answer = -2%
- Use the report as a record of your progress, and then condense it before submitting it.
- If you write down an observation, we also expect you to try and explain it.
- Questions ending with "Why?" need evidence (not just logic).
- Always use your own words (paraphrase); we will check for plagiarism.

Task 1: Improvements of Microservice 1 & 2 & 3

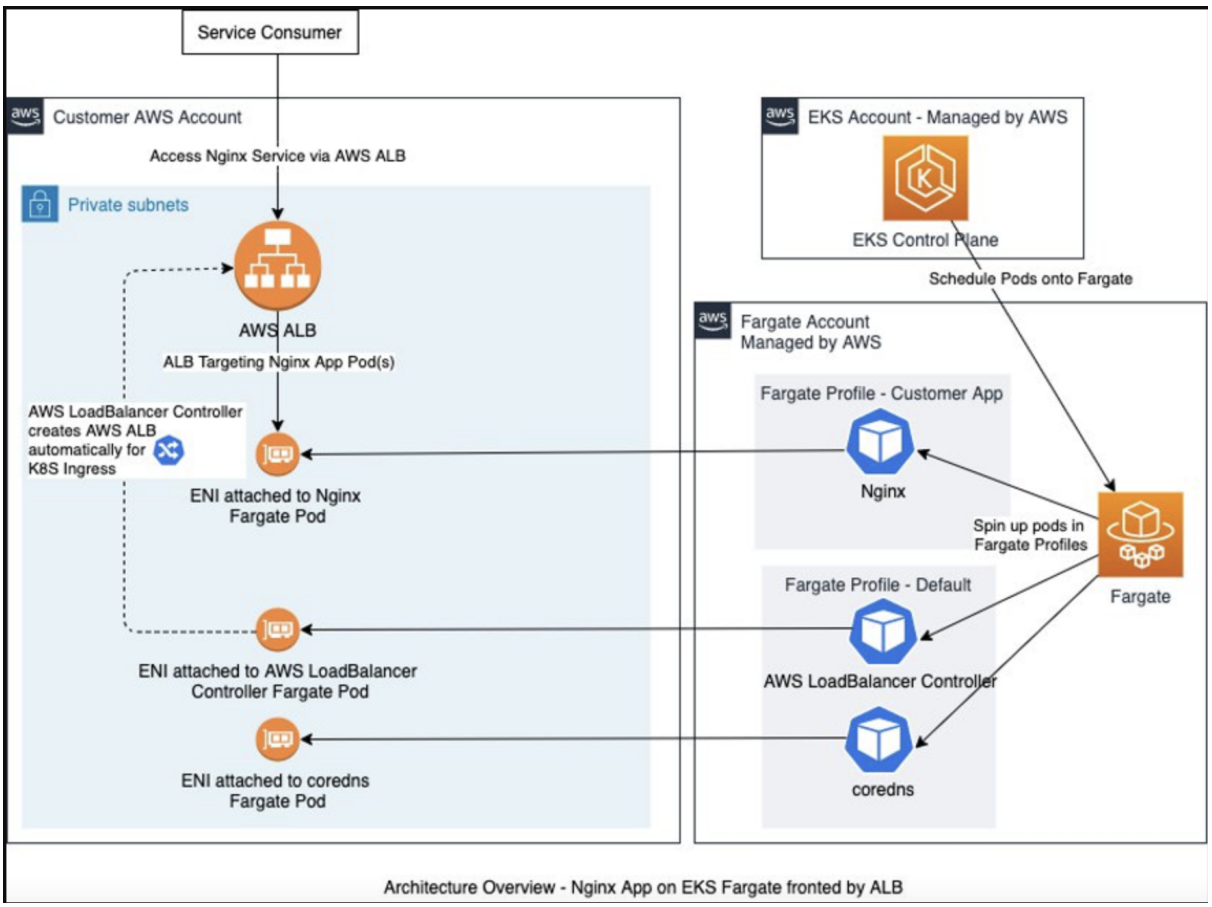
Question 1: Web-tier architecture

In Phase 3 you can choose from ECS with Fargate, EKS with Fargate and EKS with managed node groups to host your web-tier. Modernizing your application with fully-managed cloud service infrastructure is one of the biggest learning objectives of Phase 3. You have learned the concept of containers from Project 2 and built applications on self-managed Kubernetes clusters in Team Project. Now you will have the chance to explore fully-managed Kubernetes clusters.

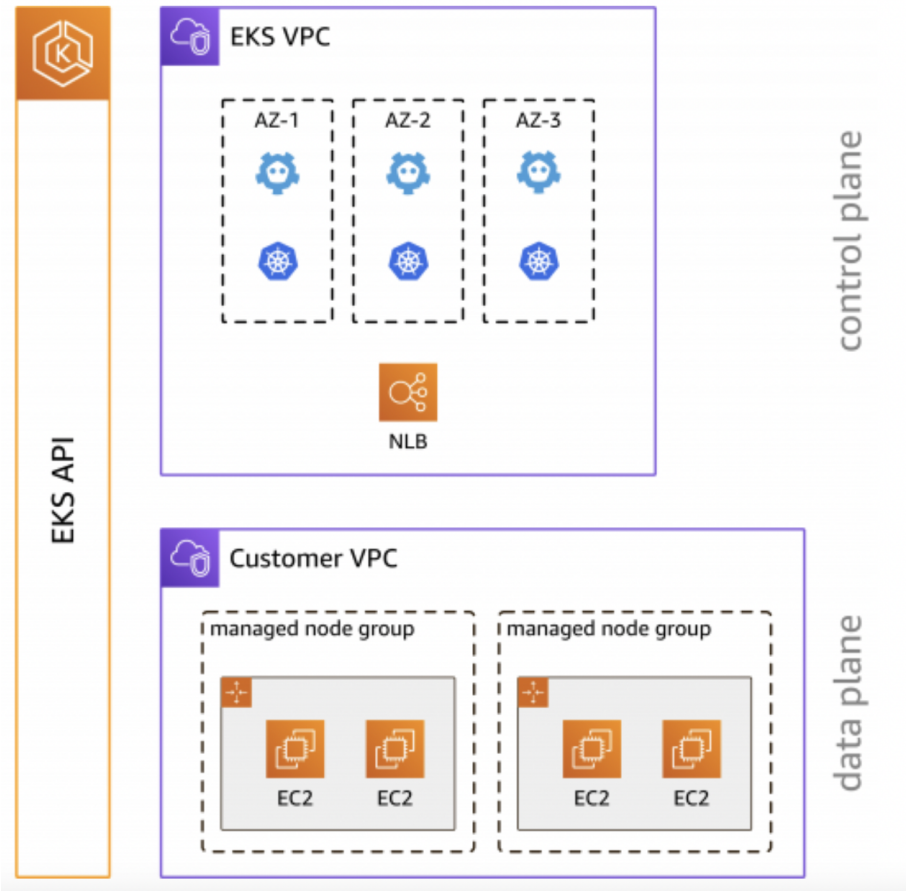
- Describe and draw graphs about the architecture of ECS with Fargate . (e.g. which pieces are involved in serving a request.)
  The application load balancer will receive requests and distribute them to Fargate pods, which are running containers.



- Describe and draw graphs about the architecture of EKS with Fargate. (e.g. which pieces are involved in serving a request.)
  The application load balancer will receive requests and distribute them to Fargate pods, which are running containers.



- Describe and draw graphs about the architecture of EKS with managed node groups. (e.g. which pieces are involved in serving a request.)
  The control plane and data plane are separate. The control plane contains the load balancer, which is involved in serving a request (forwarding requests to data plane).



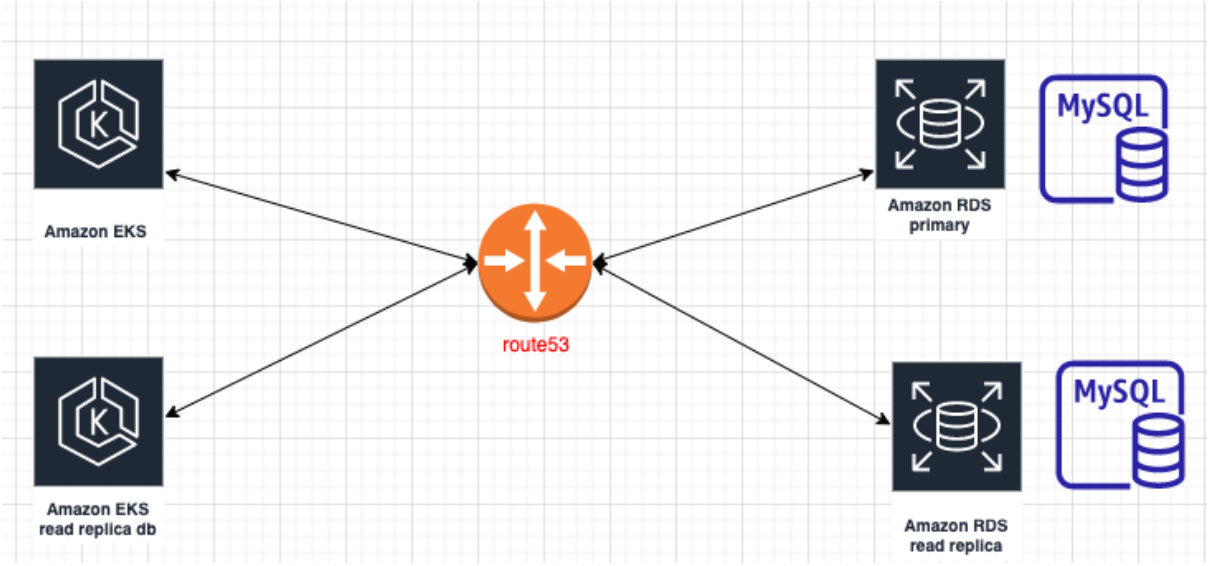
- Without using SSH to connect to the instances, how would you monitor the three architectures mentioned above?
  We can use CloudWatch to monitor Fargate groups. For EKS with managed node groups, we can also go to the "autoscaling groups" part to look at the metrics.
- Launch **all** of your microservices using either ECS or EKS with Fargate. Try to maximize your performance while making your web-tier stay under an hourly budget limit of $0.70. We want you to practice microservice architecture. That is, each container must only host one of the microservice. What is the RPS of each microservice?
  We used EKS with Fargate. The configuration is 7 pods, each with 2 vCPU and 4G memory. Total cost is 7 * (2 * 0.04048 + 4 * 0.004445) = 0.69118.
  Here is the performance of each microservice:
  m1: 29654 RPS
  m2: 22251 RPS
  m3: 6149 RPS
- Similarly, launch **all** of your microservices using EKS with managed node groups, maximize your performance while making your web-tier stay under an hourly budget limit of $0.70. What is the RPS of each microservice?
  We used 9 c6a.large instances. Total cost is (0.0765 + 20 * 0.1/720) * 8 = 0.6342
  Here is the performance of each microservice:
  m1: 54854 RPS
  m2: 49266 RPS
  m3: 8218 RPS

- Which AWS managed service did you eventually choose for the web-tier and why? (You should mention performance/cost ratio, flexibility and suitability for our use case)
  EKS managed node groups provide the best performance/cost ratio in the AWS comparison and allow different service pods to share node capacity. The same independent-service architecture is represented on AKS: Azure DevOps builds Java or ASP.NET Core images, ACR stores them, and Helm defines the workloads. NGINX routes the three public paths to internal Services. The C# projects keep m1 encoding, m2 validation/mining and m3 contact queries in separate containers.
- Describe your configuration of the web-tier during the live test and why you choose such configuration. (e.g. number of instances, instance type, vCPU, memory, replications, etc)
  EKS with managed node groups: 9 c6a.large instances, each with 20 root volume. We chose c6a.large because it's the cheapest x86 instance with 2 vCPU (excluding t-series instances). We used 9 such instances because this number fits the total budget.

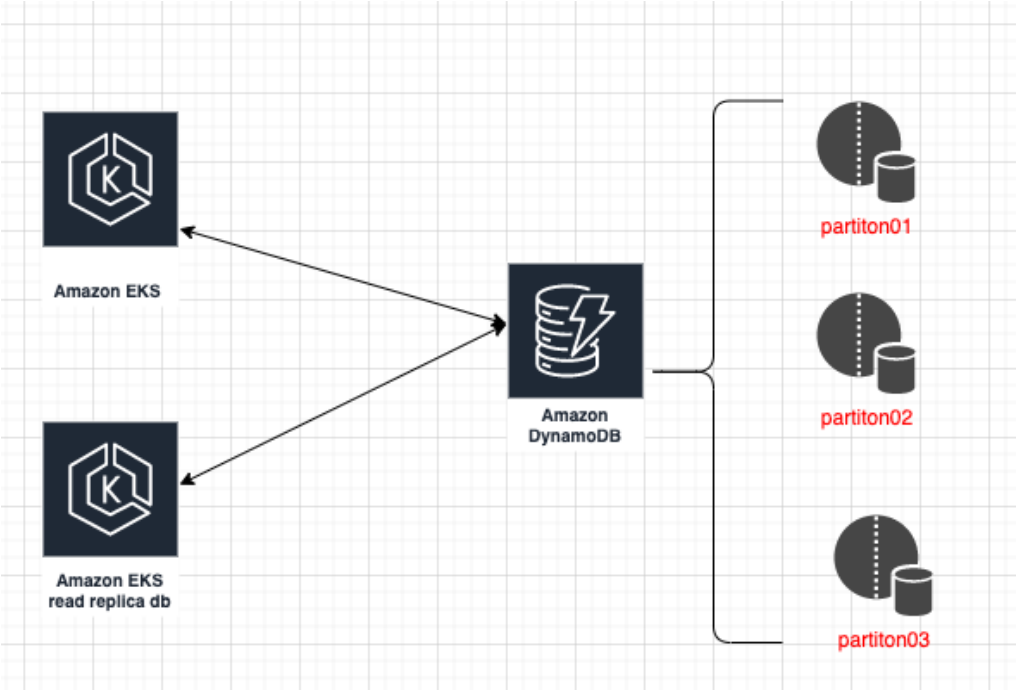
Question 2: Storage-tier architecture

In Phase 3 you can choose any database engine as long as it's supported by the designated managed services. As you've experienced in Project 3, different database engines are designed under different preferences and you need to choose one that suits Microservice 3. Various schemas can also result in a very large difference in performance! You might want to look at different metrics to understand the possible bottlenecks and think of what to optimize.

- List at least two reasonable storage-tier architecture designs using different DB engines. Explain their mechanisms of processing a Microservice 3 query. (e.g., Use graphs to show which pieces are involved in serving a request.)

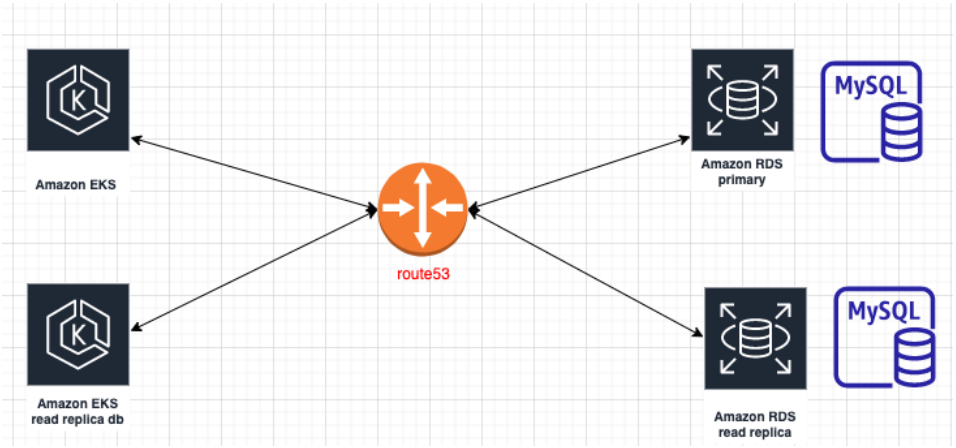


For MySQL DB, We need one route53 and two RDS MySQL db to respond to each request from EKS instances.



For DynamoDB, each request will be directly sent to the dynamo DB. However, since it is a NoSQL DB, it will further filter by the partition key and look for the correct partitions to fetch items.

- Compare at least two different storage-tiers with the same web-tier configuration and similar cost, submit your endpoint to the Sail() Platform to see the Microservice 3 performance. Which one did you eventually choose and why? (You should mention performance/cost ratio, usability)
    - We compared DynamoDB and RDS MySQL DB and finally decided to choose RDS MySQL DB. The performance of DynamoDB is similar to RDS MySQL DB. However, since the cost is higher for DynamoDB and it takes longer time for DMS to transfer from our primary data source, which is at other cloud provider, to DynamoDB at AWS. We decided not to use DynamoDB.
- Describe your final choice of managed services and the architecture for your storage-tier.
    - We decide to use EKS and RDS MySQL DB. The architecture is as below:



- What different database schemas have you designed and explored? How did the different schemas impact performance? Which one did you choose? We expect you to provide evidence that supports your decision.

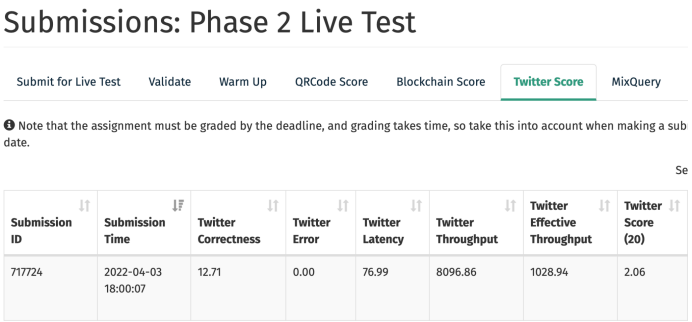
Previously we had three tables. For each request, there would be a dozen of queries for a single request.

Table Name	Table Description
TableA - users	stores users' latest information
TableB - tweets	similar to raw data, however, we eliminated tweets that are neither retweet nor reply
TableC - scores	contains interaction score and hashtag score

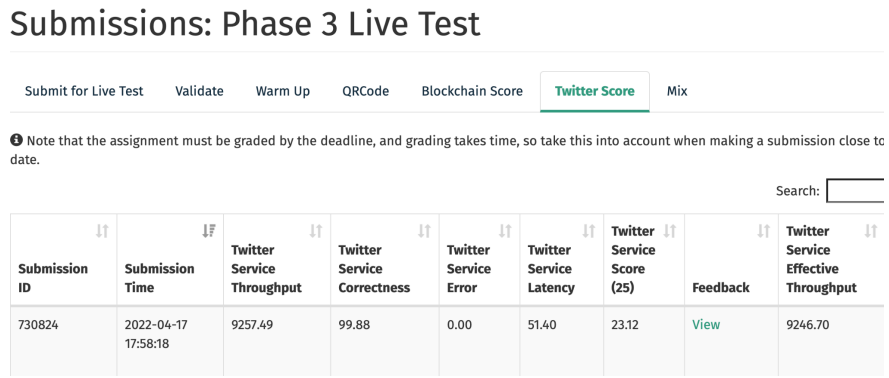
Now we only have one table and for a single request, we only do one query to db.

Table Name	Table Description
contact_tweets	One row per user_id stores contact_tweet JSON: contact users, tweets, profiles and precomputed scores. Java and C# read the same serving structure, with scoring performed after retrieval.

The RPS changed drastically. Previously we had only around 1000~ RPS.



Now we have 9246 RPS. [a round of applause for us]



Question 3: Optimizations and Tweaks

Frameworks and databases generally require tuning to reach peak performance. Each of them has tens of parameters to tune. Once you are satisfied with the schema, you can start learning about the configuration parameters to see which ones might be most relevant and gather data and evidence with each individual optimization. You have probably tried a few in Phase 1 & Phase 2. To perform even better, you probably want to continue with your experimentation.

- List as many possible items to configure or tweak that might impact performance. You need to provide at least 3 performance optimization techniques for both web-tier and storage-tier.

  For web-tier:

  1. Add cache to all services.
  2. Use a network load balancer instead of the application load balancer.
  3. Choose cheaper instances and launch more such instances.

  For storage-tier:

  1. Merge all the data needed in the web tier to a single table.
  2. Use volume with larger IOPS.
  3. Use more read replicas within the budget limit.

- Show us the benefit of your optimizations. Apply one optimization at a time and show a chart/table with their respective RPS.

  We are optimizing m3 service.

Optimization	RPS Before	RPS After
web-1	1023	1543
web-2	1543	2131
web-3	2131	3544
storage-1	3544	6583
storage-2	6583	7823
storage-3	7823	9502

- For every configuration parameter or tweak, try to explain how it contributes to performance.

  For web-tier:

  1. Reading from memory is faster than reading from the database.
  2. Network load balancer is faster than application load balancer.
  3. Launching cheaper instances can increase the total number of CPUs.

  For storage-tier:

  1. Merging all information to a single table can significantly reduce the query time; we store all the information in memory after the first query. So we can process the request from memory, which is much faster than sending requests to the database every time.
  2. Volume with larger IOPS can handle more I/O requests per second.s
  3. Using more read replicas can reduce the hotspot problem, i.e., avoid single point to be heavily loaded.

Task 2: Managed Service

Question 4: Cost and Benefit

In Phase 3, you are required to focus on both performance and cost. You have both business constraints as well as non-functional requirements. Hence, you need to calculate the cost and compare the performance of different managed services to make your design decision on the architecture of the web service.

- What is the hourly cost of the managed services you explored (list at least 4 different managed services)?
  ECS: free (pay for AWS resouces)
  EKS: $0.1 per hour (plus AWS resources)
  RDS: $0.22 per hour
  DynamoDB: $0.246 per hour
- How did you compare the performance/cost of the managed services you listed above and what kind of performance/cost results and ranking of the managed services did you get?
  We configured each service within a fixed cost (say 0.7/h for web-tier), and compared their RPS for each microservice. For web-tier, the result is: EKS with managed node groups > ECS with Fargate > EKS with Fargate. For storage-tier, the result is: RDS > DynamoDB.
- Are there any other managed services you decided not to explore? Explain your reasoning.
  We decided not to use Aurora DB because of the high cost and the confusing pricing rate.
- What are the optimizations that you have tried on the managed services and how did these optimizations affect your performance/cost results?
  For EKS with managed node groups, we chose cheaper instances with the same number of vCPUs, which saves cost and increases the total number of instances within the budget. For RDS, we built a read replica of the primary database, which increases the I/O performance.
- In Phase 1 and 2, you deployed your web-tier on self-managed Kubernetes clusters. When using fully-managed services like ECS and EKS, what are the benefits of using them in general? Which of those benefits are helpful in Phase 3? For the less useful features, can you list at least one scenario that these features become handy or even critical?
  Fully-managed services help us operate our AWS infrastructure more efficiently and securely. For example, with kops, we need to manage the master node by ourselves, including writing a configuration yaml file for the master node. With EKS, we only specify the configuration for worker nodes. This also saves cost, because the total cost for the master node (instance plus additional volume and load balancer) can be greater than $0.1/hour, which is the EKS service cost. EKS has many more features that we haven't used, such as applying security patches automatically to the control plane. This may be useful when the control plane needs to be updated frequently.

  In Phase 1 and 2, you deployed the MySQL server by yourself on self-managed Kubernetes clusters. When using managed database services like AWS RDS, what are the benefits of using these in general? Which of those benefits are helpful for Phase 3? For the less useful features, can you list at least one scenario that these features become

handy or even critical?

First of all, we don't have to manually mount any storage for the MySQL server if using RDS. However, for the self-managed MySQL server previously, we had to do that for each MySQL pod. Second, the connectivity is easy to manage and deployed. We could make the RDS public accessible when we need to ingest data from other sources (such as GCP). We used one read replica to relieve the load from the master database.

There are some features that we didn't have a chance to utilize. 1) Multi-AZ deployment. If we were working at an international company, then multi-az deployment can be useful to support fast database queries and site reliability. 2) auto-update of MySQL, there is no more need for database administration work.

Task 3: Development and Operations

Question 5: Web service deployment automation

Deployment automation covers resource provisioning, container delivery and storage connectivity.

Describe how your team automated deployment of the web tier and storage tier.

For AWS, cluster definitions and scripts configure EKS, while Terraform provisions RDS. For Azure, phase3-deployment/AzureDevOps separates infrastructure initialization from application delivery. azure-infrastructure.yml creates the resource group, ACR and AKS; azure-pipelines.yml runs Validate, Build and Deploy. The stack parameter selects dotnet, vertx or jooby.

Validation checks Shell syntax and Helm templates. Build jobs package each service, and main/master pushes Build ID-tagged images to ACR. Pull requests validate and build without pushing or deploying. Deployment creates a build-specific database Secret and uses helm upgrade --install --atomic --wait. The chart defines application Services, an NGINX gateway and environment-specific replicas: one per service for dev and three for prod. Route checks follow the rollout; rollback.sh selects a prior Helm revision.

How does the web tier connect to the storage tier?

EKS reaches RDS through private VPC connectivity and security-group rules, with Route53 providing the configured database name. AKS needs DNS and network access to an existing MySQL database containing the ETL output. A secret variable group supplies mysqlHost, mysqlUser and mysqlPassword. Application releases manage containers and configuration; database creation, import and replication remain separate storage operations.

Question 6: Live test and site reliability

Your web service in Phase 3 is evaluated similarly to how we evaluated your web service in Phase 2. In Phase 3, you can make as many attempts as you need by the deadline. However, all these attempts do not contribute to your score. Your team's Phase 3 score is determined solely by the live test. The live test is a one-time test of your web service during a specified period of

time, so you will not want anything to suddenly fail; in case you encounter failure, you (or some program/script) probably want to notice it immediately and react!

- How would you monitor the health of your system (both web-tier and storage-tier)?

  For AWS, CloudWatch tracks RDS burst credits, CPU, available memory, I/O and queue depth, together with web-node CPU load. Kubernetes pod status and logs apply to both EKS and AKS. The Helm chart defines TCP startup, readiness and liveness probes for applications and an HTTP /healthz check for NGINX. Database queries require separate monitoring because a listening application port does not establish storage health.

- What metrics helped you understand the performance of your web service during the live test?
  - IO Burst Credit, CPU Utilization Percentage, Available Memory, IO counts, and Queue Depth.
- What other monitoring or data collection tools did you use? What did the statistics tell you?
  - We used Kubernetes Dashboard to monitor the status of the cluster such as available memory, CPU usage, and the health of pods and deployments.
- Should something crash in your system, will your service still be able to serve requests? What have you done to resolve these issues? Will this affect your overall performance? Think about this in different scenarios, such as web-tier, storage-tier and so on.

  Multiple web replicas allow healthy pods to continue serving during a container failure. Helm uses --atomic --wait for failed rollouts; a later smoke-check failure requires a separate rollback decision. Database recovery is managed independently from application releases. The C# m3 handler logs database errors and returns HTTP 503. Replicas reduce single-instance disruption, but database or network failures can still affect request availability.

Task 4: Reflection

Question 7: Consistency and Scalability

In this team project, we did not ask you to respond to any write requests. Think about whether your service could provide the same quality under a load of write requests or a growing amount of data.

- What problems might arise if your service served write requests? What are some problems that you might encounter and how will you change your design to address them?

  For a single object, e.g, a data entry in the database, we can't allow multiple users to write to this object at the same time. So we should implement a write lock mechanism to prevent concurrent writing. This can be either implemented by using a write lock or copying the object every time when a write request comes. Using a write lock is simple but this will significantly reduce the throughput. So we may change our design to allow copy for data entry and implement a finally-consistent model.
- For only read requests, would your design work as well if the quantity of data was 10 times larger? What about 100x? Why or why not? (Assume you get a proportional amount of machines.)

For only read requests, our design will work because we have many read replicas proportional to the number of read requests. So the more replicas we have, the less likely we would encounter a hotspot problem.

Question 8: Postmortem

- How did you set up your local development environment? How was the development process using managed services different from what you did in Phase 2?
  The development toolchain includes IntelliJ/Maven for Java, the .NET 8 SDK for C#, Docker, Helm and cloud CLIs. AWS deployment uses eksctl; Azure Pipelines packages the chosen stack and releases to the configured AKS environment.
- Did you attempt to generate a load to test your system on your own? If yes, how? And why?
  Yes. In order to profile the Java application and resolve the bottleneck,we used JMeter to generate loads.
- Describe an alternative design to your system that you wish you had time to try.
  We could have also tried to utilize NoSQL databases such as HBase or MongoDB to store tweets information.
- Which were the toughest roadblocks that your team faced in Phase 3 especially for managed services?
  The most challenging part in Phase 3 is to configure all managed services to make them work correctly and efficiently, including connecting to the RDS storage-tier from the EKS web-tier, and automating the deployment of both tiers.
- Did you do something unique (any cool optimization/trick/hack) that you would like to share?
  We changed some configurations in *my.cnf* of MySQL database to help accelerate loading data into MySQL database. Otherwise, it might take days to load a large table into the database.

Task 5: Summary

Question 9: Summary

After Phase 3, you are ready to make important decisions and choices about SQL vs NoSQL, trade-offs between different Cloud Service Providers, sharding vs replication, MapReduce vs Spark, self-managed vs fully-managed services, etc. as you go out to industry. Please answer the following questions based on your experience in phases 1, 2, and 3.

- Which Cloud Service Provider did you use for ETL? Why?
  We used GCP because GCP Zeppeline is easy to configure and use, and the budget is adequate.
- Which programming framework did you use for ETL (e.g., MapReduce, Spark, other)? Why?
  We use Spark for ETL. Spark is a wonderful candidate for iterative big data processing. We tried to use Java for ETL, but it's very slow and memory sensitive compared to Spark. We pre-compute the time for java ETL, it will take hundreds of hours to finish, so we just gave it up. Spark also supports Spark SQL, which is very convenient to use.
- Which programming framework did you use for web-tier (e.g., Spring, Flask, other)?

Why?

The web tier includes Java with Jooby/Vert.x and C# with ASP.NET Core Minimal APIs. Vert.x is used for the reported AWS results. ASP.NET Core separates the three service projects and provides built-in JSON, compression and cryptographic APIs, with MySqlConnector for m3.

Which kind of database did you use for Microservice 3, and why?

MySQL supports indexed lookup and the per-user JSON serving schema consumed by both stacks. It fits the ETL output and our SQL experience. HBase offers a row-key-oriented alternative, but requires a different access model and key design.

- Compare the entire architecture of your web service in phase 2 and phase 3 in terms of performance/cost ratio, scalability, fault tolerance and the effort to deploy, maintain and monitor the service.

	Phase2	Phase3
performance	low	high, because we change the database schema, which leads to higher performance
p/c ratio	medium	higher performance/cost ratio
scalability	the same as Phase3	the same as Phase2
failt tolerance	cache may lead to out of memory error	remove cache to avoid possible failure
monitor	CloudWatch	CloudWatch + Kubenetes Dashbord

- What is the coolest optimization/hack/learning you have achieved throughout the team project? (eg. Cost, Scalability, Performance, etc.)

  The most useful optimization is aligning the serving schema with the response. Repeated queries inside a contact loop add latency even when the database is close to the web tier. Storing the required contact data in one per-user JSON record supports a single retrieval followed by in-memory computation.

  The C# ContactRepository expresses this access pattern with a parameterized user_id query and an asynchronous pooled connection. TwitterService merges reply/retweet records, counts overlapping phrase matches, matches complete hashtags without case sensitivity, and multiplies interaction, hashtag and keyword scores. Results sort by score and then numeric user ID, both descending. Keeping request data local avoids sharing mutable state across concurrent handlers. The broader lesson is to reduce unnecessary database work before tuning framework or infrastructure parameters.